

Architektur und Umsetzung eines exemplarischen Online-Stores "Das Holzwerk" Leading-Edge-Development

Projektbericht

Gegenstand:	Entwurf, Implementierung und Qualitätssicherung eines verteilten Onlineshops mit nachvollziehbaren Bestell- und Kompensationsabläufen
Technologiestand:	.NET10, gRPC, RabbitMQ, PostgreSQL, OpenTelemetry, OpenAPI 3.1
Dokumentstand:	31. August 2026
Autor:	B.Sc. P. Förderer
E-Mail:	p.foerderer@fh-swf.de

Dokumentübersicht

Inhaltsverzeichnis

Dokumentübersicht	2
1 Einleitung	3
1.1 Problemstellung	3
1.2 Ziele	3
1.3 Abgrenzung	3
2 Grundlagen	4
2.1 Microservices und Gateway	4
2.2 Fassade und Provider-Strategien	4
2.3 SAGA statt verteilter ACID-Transaktion	4
2.4 Event Sourcing Light und Beobachtbarkeit	4
2.5 Synchron und asynchron	5
3 Systemarchitektur	5
3.1 Komponenten und Verantwortlichkeiten	5
3.2 Interaktionen im Happy Path	6
3.3 Schnittstellen und Betriebsmodell	6
4 Implementierung	7
4.1 Payment-Fassade	7
4.2 AuditSnapshot-Schema	8
4.3 Kompensationslogik	8
4.4 Fault Injection und Präsentationsmodus	9
5 Qualitätssicherung	9
5.1 Teststrategie	9
5.2 Automatisierte Testergebnisse	10
5.3 Nachweise der Fehlerszenarien	10
5.4 Bewertung und verbleibende Testlücken	10
6 Reflexion	11
6.1 Gute Entscheidungen	11
6.2 Weniger gute Entscheidungen	11
6.3 Eingegangene Kompromisse	11
6.4 Fazit	12
Literatur und Standards	13

Kurzfassung

Der Holzwerk-OnlineStore verbindet eine öffentliche REST-Fassade mit internen gRPC-Services. RabbitMQ entkoppelt Rechnungserstellung und Auditing; PostgreSQL speichert append-only Audit-Snapshots. Eine Payment-Fassade kapselt austauschbare Provider. Der ShopService orchestriert den Bestellprozess als SAGA und kompensiert Zahlungen und Reservierungen bei Folgefehlern.

Merkmal	Ausprägung
System-	Browserzugriffe ausschließlich über den StoreProxy; interne Dienste über gRPC und RabbitMQ
Qualitätsfokus	Erweiterbarkeit, Nachvollziehbarkeit, Fehlertransparenz, reproduzierbarer lokaler Betrieb
Persistenz	PostgreSQL für Audit-Snapshots; JSON-Dateien für Lager- und Rechnungsdaten der Demonstrationsumgebung
Nachweis	28 automatisierte Tests sowie End-to-End-Prüfungen der vier deterministischen Präsentationsszenarien

1 Einleitung

1.1 Problemstellung

Ein Bestellprozess wirkt aus Sicht des Kunden wie eine einzelne Aktion. Intern müssen jedoch Bestand, Zahlung, Rechnung und Protokollierung koordiniert werden. In einem verteilten System können diese Teilschritte unabhängig voneinander fehlschlagen oder zu spät antworten. Eine lokal erfolgreiche Zahlung bedeutet beispielsweise noch nicht, dass der Lagerbestand endgültig ausgebucht oder eine Rechnung erstellt wurde. Klassische Datenbanktransaktionen über alle Systemgrenzen hinweg wären technisch aufwendig, würden die Komponenten eng koppeln und passen nicht zu einem ereignisbasierten Rechnungs- und Auditingpfad.

Das Projekt adressiert daher drei miteinander verbundene Fragen:

- Wie lässt sich der Bestellprozess in fachlich getrennte Services zerlegen, ohne die Konsistenzanforderungen zu verletzen?
- Wie werden Fehler und Timeouts so behandelt, dass Kunde und Betreiber einen eindeutigen Zustand erkennen können?
- Wie bleibt die Historie eines Vorgangs über synchrone Aufrufe, Nachrichten und Kompensationen hinweg nachvollziehbar?

1.2 Ziele

Das Ergebnis soll eine lokal ausführbare Referenzimplementierung eines Onlineshops sein. Im Vordergrund stehen Architektur und Verhalten. Da es sich hier um ein exemplarisches Lernbeispiel handelt, wurden einige Freiheiten, wie zB. Payment-Provider Implementierung als Stub's oder ein generiertes Frontend, bereits in der Voraussetzung gewährt.

Daraus ergeben sich folgende Ziele:

1. Eine öffentliche, ressourcenorientierte REST-API mit OpenAPI-3.1- Beschreibung und einheitlichen Problem-Details nach RFC 9457
2. Getrennte Verantwortlichkeiten für Orchestrierung, Lager, Zahlung, Rechnung und Auditing
3. Eine konfigurierbare Payment-Fassade mit mehreren Providern und einem Erweiterungspunkt nach dem Open/Closed Principle
4. Ein expliziter SAGA-Ablauf mit definierten terminalen Zuständen und nachvollziehbaren Kompensationen
5. Eine unveränderliche, korrelationsbezogene Audit-Historie in PostgreSQL sowie ergänzende technische Logs und Telemetrie
6. Reproduzierbarer Betrieb über PowerShell-Skripte und eine vorführbare Fault Injection pro Bestellung

1.3 Abgrenzung

Die Implementierung ist eine Entwicklungs- und Demonstrationsumgebung. PayPal, Stripe und DemoPay sind lokale Stubs und bewegen kein echtes Geld. Die Rechnungsmail wird standardmäßig in eine Pickup-Outbox geschrieben; ein produktiver SMTP-Versand, Webhooks externer Provider, Authentifizierung, Autorisierung, Hochverfügbarkeit und Backupkonzepte sind nicht oder nur zum Teil, Gegenstand des Projekts. Nur Audit-Snapshots benötigen relationale Persistenz nach Voraussetzung. Lager- und Rechnungsdaten bleiben bewusst dateibasiert. Diese Abgrenzung verhindert, dass Infrastrukturthemen die Untersuchung der Architektur- & Entwurfsmuster überlagern.

Auch das Frontend (index.html, css- & js-Files) durfte vernachlässigt werden und wurde durch ChatGPT's-Codex-Plattform^[1] generiert, um den Fokus nicht auch auf Webdesign und JavaScript zu legen.

2 Grundlagen

2.1 Microservices und Gateway

Die Services orientieren sich an fachlichen Verantwortlichkeiten. Der StoreProxy ist der einzige öffentliche Einstiegspunkt und leitet explizit freigegebene HTTP-Ressourcen an den ShopService weiter. Dadurch kennt der Browser weder interne Ports noch gRPC-Verträge. Der ShopService besitzt die Prozesssicht, während Warehouse-, Billing-, Invoice- und AuditService ihre jeweiligen Fachfunktionen kapseln. Diese Trennung unterstützt unabhängige Fehlerbehandlung und vermeidet direkte Zugriffe der Website auf interne Komponenten. Während die YARP-Middleware hier, zentrales Routing und LoadBalancing übernimmt.

Die öffentliche API folgt dem Ressourcenstil: Bestellungen werden mit `POST /api/orders` erzeugt, Rechnungen über `GET /api/invoices/{invoiceId}/pdf` abgerufen und Audit-Historien über `GET /api/order-audits/{correlationId}/snapshots` gelesen. Aktionsnamen wie `/chargeOrder` oder `/getAudit` werden vermieden. OpenAPI 3.1 beschreibt Requests, Antworten, Statuscodes und Fehlerfälle. Problem-Details nach RFC 9457 liefern neben `type`, `title`, `status`, `detail` und `instance` stets eine `correlationID` [2, 3].

2.2 Fassade und Provider-Strategien

Das Fassadenmuster bietet einem Client eine vereinfachte Schnittstelle zu einem komplexeren Teilsystem [4]. Im Projekt ist die `PaymentFacade` der einzige Zugriffspunkt auf konkrete `IPaymentProvider`-Implementierungen. Sie vereinheitlicht die Operationen `ChargeAsync`, `RefundAsync` und `GetStatusAsync`, validiert die Providerwahl und behandelt Zeitüberschreitungen an einer Stelle.

Die konkreten Adapter bilden quasi austauschbare Strategien im Sinne des Entwurfsmusters, PayPal und Stripe sind initial aktiviert; DemoPay bleibt registriert, ist jedoch deaktiviert. Die Website trifft keine Vorauswahl. Der Kunde wählt den Provider für genau eine Bestellung, und der BillingService prüft diese Wahl erneut gegen die Konfiguration. Diese Konfiguration kann auch über die Datei: `appsettings.json` des Projekts `BillingService`, zB. auch als Umgebungsvariable gesetzt werden. Zur Kenntlichmachung der zusätzlichen Konfigurationsebene wurde DemoPay beispielhaft deaktiviert. Neue Adapter werden per Tyediscovery registriert. Deshalb müssen weder Fassade noch Startcode verändert werden, solange der neue Typ die Schnittstelle `IPaymentProvider` implementiert.

2.3 SAGA statt verteilter ACID-Transaktion

Das SAGA-Pattern (dt. *Sage, Erzählung*) zerlegt einen Geschäftsprozess in lokale Transaktionen und ordnet erfolgreichen Schritten passende Gegenaktionen zu [5]. Im vorliegenden System orchestriert der ShopService diese Reihenfolge. Eine Bestandsreservierung wird bei abgelehnter Zahlung freigegeben. Ist die Zahlung bereits erfolgreich und scheitern anschließend Event-Publikation oder Lager-Commit, wird zuerst ein Refund über die Fassade versucht und danach die Reservierung storniert.

Kompensation ist nicht mit einem technischen Rollback gleichzusetzen. Sie ist eine neue fachliche Operation, kann selbst fehlschlagen und muss deshalb einen eigenen Audit-Zustand besitzen. Der Endzustand `ROLLBACK_COMPLETED` bedeutet, dass alle vorgesehenen Gegenaktionen erfolgreich waren, wohingegen `ROLLBACK_FAILED` betriebliche Probleme signalisiert.

In Abgrenzung zu typischen ACID-Transaktionen (*Akronym aus: Atomicity, Consistency, Isolation, Durability*) wie sie auch gerne bei Datenbanktransaktionen verwendet werden, kann innerhalb verteilter nebenläufiger Systeme keine *strenge* Konsistenz erzwungen werden, daher bietet das SAGA-Pattern eine abgeschwächte Form, der „eventuellen Konsistenz“.

2.4 Event Sourcing Light und Beobachtbarkeit

Vollständiges Event Sourcing rekonstruiert den aktuellen Aggregatzustand aus dem Ereignisstrom und verwendet Ereignisse als primäre Persistenz [9]. Der OnlineStore verfolgt bewusst eine leichtere Variante: fachlich relevante Zustandsübergänge werden als append-only Audit-Snapshots gespeichert, die operativen Modelle von Lager, Zahlung und Rechnung werden jedoch nicht daraus wiederhergestellt. Der Begriff *Event Sourcing Light* bezeichnet hier somit eine unveränderliche, geordnete Historie, nicht ein vollständiges Event-Sourcing-System.

Auditing und technisches Monitoring bleiben getrennt. Audit-Snapshots erklären, *was fachlich geschah*; JSONL-Logs, Traces und Metriken erklären, *wie sich die Software technisch verhielt*. Eine durchgängige Correlation-ID verbindet HTTP-Request, gRPC-Aufrufe, RabbitMQ-Nachrichten, Logeinträge und Datenbankzeilen.

2.5 Synchron und asynchron

Bestandsreservierung, Zahlung und Lager-Commit beeinflussen die unmittelbare Bestellantwort und werden synchron per gRPC ausgeführt. Rechnungserstellung und Auditing sind zeitlich entkoppelbar und laufen über RabbitMQ. Durable Queues, persistente Nachrichten, Publisher-Confirms, manuelle Acknowledgements und Dead-Letter-Queues ermöglichen eine At-least-once-Verarbeitung [10]. Daraus folgt Eventual Consistency: Eine Bestellung kann bereits abgeschlossen sein, während Rechnung oder Audit-Snapshot noch verarbeitet werden.

3 Systemarchitektur

3.1 Komponenten und Verantwortlichkeiten

Komponente	Port	Verantwortung
StoreProxy	6680	Öffentlicher Einstieg, Website, YARP-Routing, Rate Limits und Proxy-Fehlerbehandlung
StoreBackend	6681	Dateibasierte Produkt-, Bestands- und Reservierungspersistenz
ShopService	6682	REST-Fassade, Readiness-Prüfung und SAGA-Orchestrierung
WarehouseService	6683	Katalog, Reservieren, Ausbuchen und Freigeben
BillingService	6684	Payment-Fassade, Providerwahl, Charge, Refund und Zahlungsereignis
InvoiceService	6685	Asynchrone Rechnung, PDF und E-Mail-Pickup-Outbox
AuditService	6686	Konsum, Validierung, Persistenz und Abfrage von Audit-Snapshots
OTel Collector	6687	Zentrale Annahme technischer Logs, Traces und Metriken
PostgreSQL	6688	Append-only Persistenz der Audit-Historie
RabbitMQ	5672	Transport von Zahlungs- und Audit-Ereignissen

Tabelle 1: Laufzeitkomponenten der lokalen Umgebung

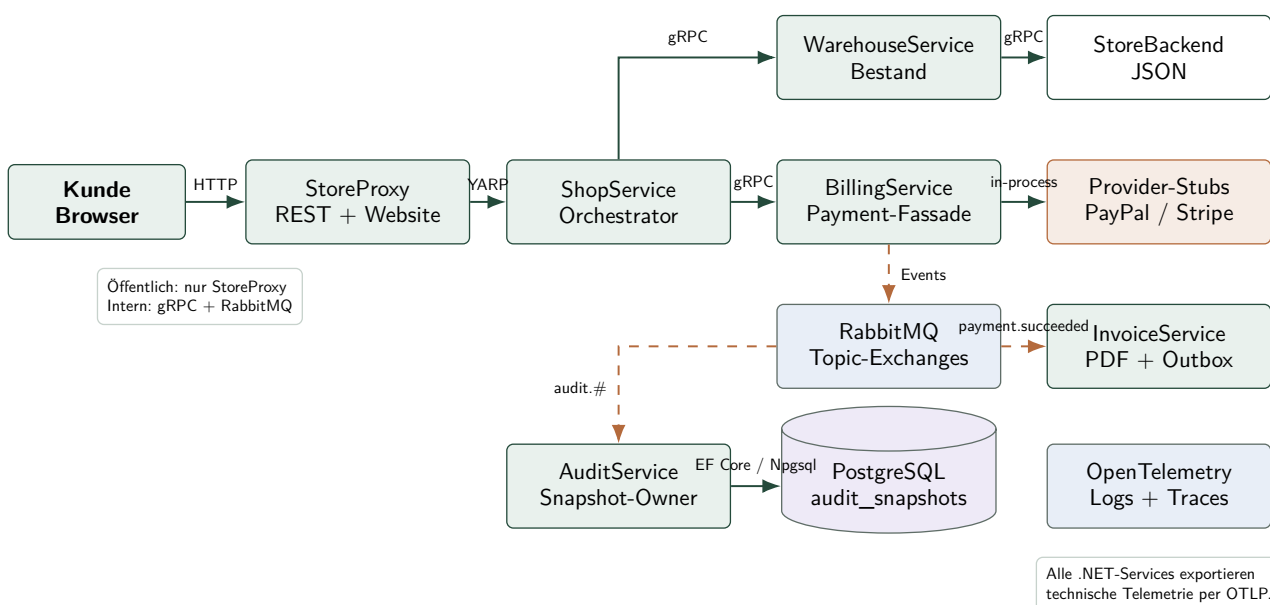


Abbildung 1: Systemkontext und zentrale Kommunikationsbeziehungen

3.2 Interaktionen im Happy Path

Der erfolgreiche Ablauf folgt sieben fachlichen Schritten. Die Correlation-ID wird beim Eingang erzeugt und zugleich als `orderId` verwendet:

1. Der StoreProxy leitet `POST /api/orders` weiter; der ShopService prüft Betriebsbereitschaft und Request
2. Der WarehouseService reserviert die Artikel persistent im StoreBackend
3. Der BillingService delegiert die Zahlung über die Payment-Fassade an den ausgewählten Provider
4. Der Provider bestätigt; der BillingService protokolliert das Ergebnis
5. Der BillingService publiziert `payment.succeeded`; der InvoiceService erstellt die Rechnung asynchron
6. Der WarehouseService bucht die reservierten Waren endgültig aus
7. Der ShopService schreibt `ORDER_COMPLETED` und antwortet mit `201 Created`. Er wartet nicht auf das fertige PDF

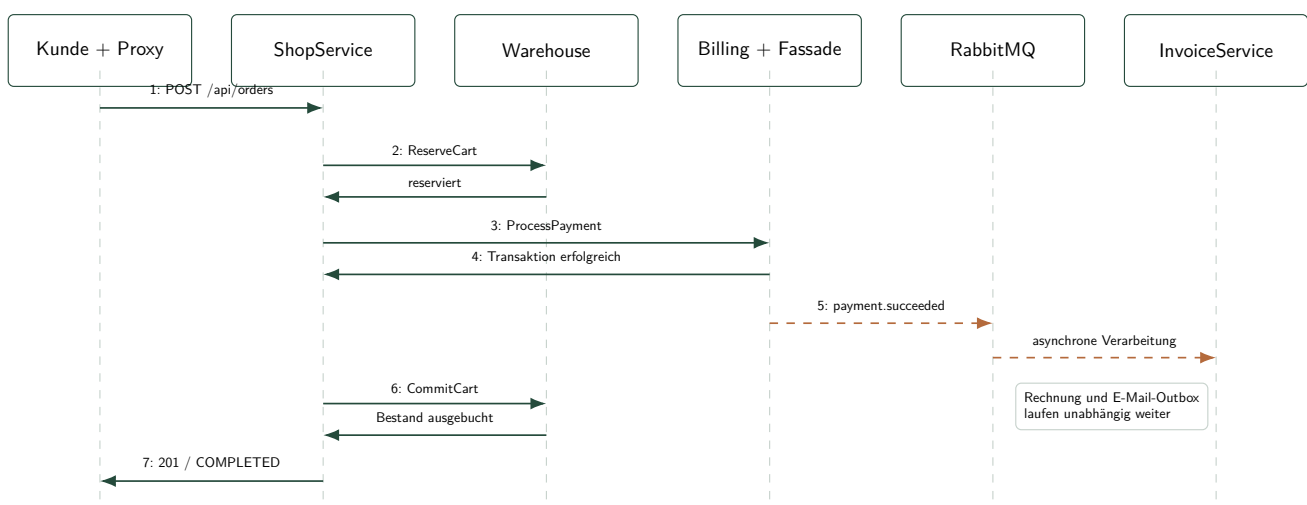


Abbildung 2: Happy-Path-Sequenz der implementierten Bestelllogik

3.3 Schnittstellen und Betriebsmodell

Die internen Verträge der gRPC-Services, liegen als typische Protobuf-Dateien vor. Der öffentliche StoreProxy kennt diese Verträge aber nicht, sondern routet ausschließlich HTTP zum ShopService. YARP prüft dessen Health-Endpunkt aktiv, setzt eine kanonische `X-Correlation-ID` und übersetzt Weiterleitungsfehler in Problem-Details. Rate Limits begrenzen insbesondere Bestellungen auf zehn Requests pro Client und Minute. Der Checkout selbst wird wegen möglicher Zahlungswirkungen nicht automatisch wiederholt.

Die Verwendung der YARP-Middleware [8] ermöglicht völlig nativ, die Webanwendung zu routen (per ReverseProxy und Layer7-Routing), LoadBalancing zu nutzen, Health-Checks durchzuführen sowie die SSL/TLS-Zertifikatsverwaltung zu übernehmen (HTTP2.0).

Das Startskript `Start-VSTOnlineStore.ps1` verwaltet PostgreSQL, OpenTelemetry Collector und alle .NET-Services komfortabel und in definierter Reihenfolge. PostgreSQL wird beim Stop zuletzt kontrolliert beendet. RabbitMQ bleibt eine externe aber notwendige Komponente und muss installiert oder per Docker gehostet werden.

Einzelne Komponenten können unabhängig gestartet und gestoppt werden; dadurch lassen sich Ausfälle und Wiederanlauf gezielt untersuchen.

Website, Services und Komponenten können aber auch einzeln populiert und zur Ausführung gebracht werden, davon wurde aber unter dem Gesichtspunkt der Präsentationsfähigkeit (Live-Demo) Abstand genommen, aber dennoch über den GIT-Ordner `ReleaseBuild`, inklusive Anleitung für das manuelle Setup, berücksichtigt.

4 Implementierung

4.1 Payment-Fassade

Der gRPC-Service `BillingOperationsGrpcService` ist die fachliche Servicegrenze, der konkrete Provider wird ausschließlich hinter der Fassade aufgerufen. Die Schnittstelle bietet für alle Adapter dieselben drei Operationen:

Listing 1: Vereinfachter Providervertrag

```
public interface IPaymentProvider
{
    Task<PaymentChargeResult> ChargeAsync(
        Guid orderId, long amountInCents, string currency,
        CancellationToken cancellationToken);

    Task<PaymentRefundResult> RefundAsync(
        string transactionId, long amountInCents,
        CancellationToken cancellationToken);

    Task<PaymentStatusResult> GetStatusAsync(
        string transactionId, CancellationToken cancellationToken);
}
```

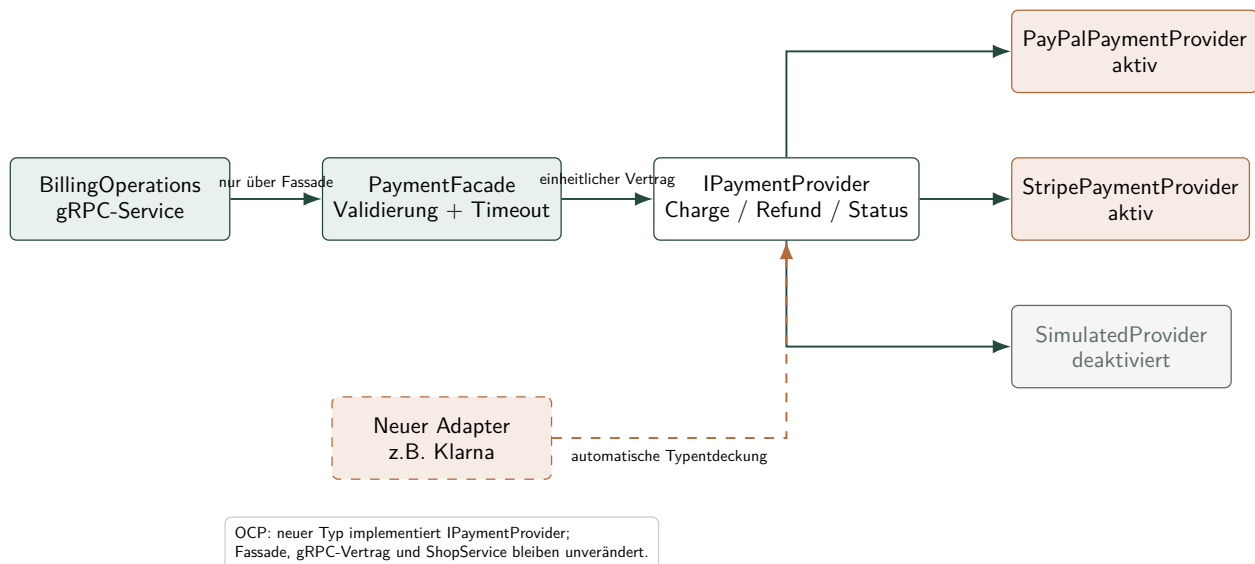


Abbildung 3: Payment-Fassade und Erweiterung um einen Provider

`EnabledProviderKeys` steuert, welche registrierten Adapter für neue Bestellungen zulässig sind. `ActiveProviderKey` dient nur als serverseitiger Fallback für ältere oder interne Aufrufer. Ein Timeout wird zentral mit einem verknüpften `CancellationToken` umgesetzt und für jeden Provider in dieselbe Fehlersemantik übersetzt.

Für Refund und Status merkt sich die Fassade den Provider einer Transaktion in einem nebenläufigen Dictionary. Dadurch bleibt der Aufrufer providerneutral. Diese Zuordnung ist gegenwärtig prozesslokal; ihre Persistierung ist eine bewusste offene Anforderung und wird in Kapitel 6 kritisch bewertet.

4.2 AuditSnapshot-Schema

Jeder fachliche Zustand wird als `AuditSnapshot` abgebildet. Die Anwendung erzeugt zunächst einen Entwurf. Der `AuditService` ergänzt `previousEventID` und eine globale `sequenceNumber` atomar:

Listing 2: *Logisches AuditSnapshot-Schema*

```
{
  "eventID": "uuid",
  "correlationID": "uuid",
  "eventType": "PAYMENT",
  "responsibleService": "ShopService",
  "timestamp": "2026-08-25T10:15:22.184Z",
  "payload": {
    "phase": "PAYMENT_REFUND_REQUESTED",
    "orderStatus": "IN_PROGRESS"
  },
  "previousEventID": "uuid | null",
  "actor": "ShopService",
  "statusCode": "COMPENSATING",
  "sequenceNumber": 4711
}
```

PostgreSQL verwendet native `uuid`- und `timestamp with time zone`-Typen; der Payload liegt als `jsonb` vor. Primär-, Fremd-, Unique- und Check-Constraints sichern Struktur und Verkettung. Ein *transaktionsgebundener Advisory Lock* serialisiert nur Ereignisse derselben Correlation-ID, und vermeidet gleichzeitig die Sperrung von Tabellen. Datenbanktrigger lehnen `UPDATE`, `DELETE` und `TRUNCATE` ab. Damit wird Append-only nicht nur als Anwendungskonvention, sondern auch in der Datenbank durchgesetzt [11].

RabbitMQ kann Nachrichten im At-least-once-Modell erneut zustellen. Die `eventID` wirkt deshalb als Idempotenzschlüssel. Ein bereits vorhandenes, inhaltlich identisches Ereignis gilt als erfolgreiche Wiederholung; ein widersprüchliches Duplikat wird abgelehnt. Nur der `AuditService` besitzt das Schema. Andere Services publizieren Ereignisse und lesen Historien über die interne gRPC-Schnittstelle des `AuditService`.

4.3 Kompensationslogik

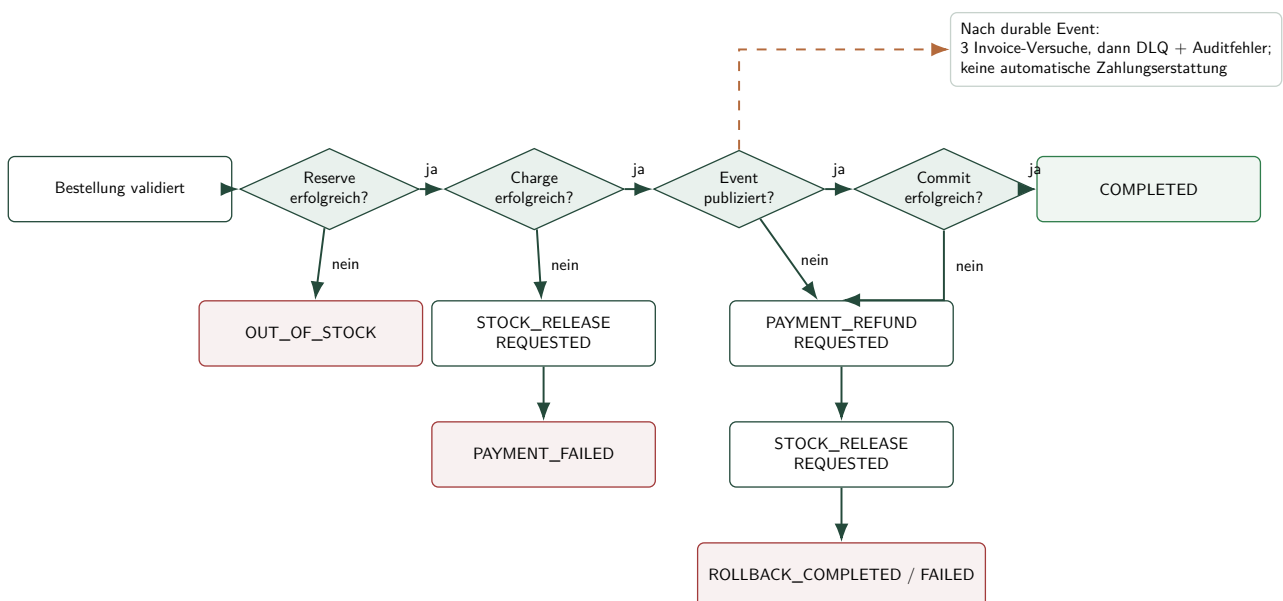


Abbildung 4: *SAGA-Entscheidungen und fachliche Gegenaktionen*

Jeder Gegenlauf beginnt mit einem eigenen Snapshot im Status `COMPENSATING`, der Erfolgsfall wird mit `COMPENSATED` und ein Fehler mit `FAILURE` gespeichert. Eine Zahlungsablehnung führt nur zur Lagerfreigabe, weil noch keine erfolgreiche Zahlung existiert. Nach erfolgreicher Zahlung werden bei fehlgeschlagener Event-Publikation oder fehlgeschlagenem Commit, Refund und Lagerfreigabe, nacheinander ausgeführt. Ein technisch nicht bestätigter Commit wird genau einmal *idempotent* wiederholt, bevor die Kompensation beginnt.

Ist `payment.succeeded` hingegen bereits dauerhaft im Broker angenommen, gehört die Rechnungserstellung dennoch zur asynchronen Verarbeitung und wird bei Fehlschlag, maximal dreimal nacheinander ausgeführt. Die Phasen `INVOICE_RETRY_SCHEDULED`, `INVOICE_RETRY_EXHAUSTED` und `INVOICE_PROCESSING_FAILED` werden auditiert; anschließend gelangt die Nachricht in die DLQ (*engl. Dead Letter Queue*). Die Zahlung wird in diesem Fall nicht rückgängig gemacht, weil der Bestellabschluss bereits fachlich bestätigt und die Nachricht dauerhaft vorhanden ist.

4.4 Fault Injection und Präsentationsmodus

Der Schalter `-PresentationMode` aktiviert vier deterministische, bestellungsbezogene Szenarien. Die Website zeigt sie im Warenkorb an und überträgt den Schlüssel über REST, gRPC und RabbitMQ. Im normalen Start ist der Modus deaktiviert; eine unbeabsichtigte globale Störung wird damit vermieden.

Szenario	Erwarteter Ablauf	Endzustand
Zahlung abgelehnt	Reservierung freigeben; kein Rechnungsevent	<code>PAYMENT_FAILED</code>
Bestand fehlt	Nach Reserve-Ablehnung keine Zahlung und keine Rechnung	<code>OUT_OF_STOCK</code>
Invoice nicht verfügbar	Drei Versuche, Fehler-Snapshots und DLQ; kein Refund	Bestellung <code>COMPLETED</code> , Invoice fehlerhaft
Warehouse-Commit fehlerhaft	Refund über Fassade, danach Reservierung wieder freigeben	<code>ROLLBACK_COMPLETED</code>

Tabelle 2: Deterministische Vorführenszenarien

Mit der Aktion `PresentationSnapshots` werden alle Snapshots einer Correlation-ID per `ShopService` über die öffentliche Audit-Ressource gelesen (`GET http://localhost:6680/api/order-audits/correlationId/snapshots`), formatiert in `Logs/Presentation` abgelegt und in Notepad++ geöffnet. Damit zeigt die Präsentation nicht nur die sichtbare Fehlerreaktion, sondern auch den internen Kompensationslauf.

5 Qualitätssicherung

5.1 Teststrategie

Die Testpyramide konzentriert schnelle Unit-Tests auf fachliche Regeln und Integrationstests auf die Orchestrierung:

- **Payment-Fassade:** Erfolg, Ablehnung, Timeout je Provider, Konfiguration, Providerwahl, deaktivierter Adapter, Refund, Status und automatische Adaptererkennung.
- **StoreBackend:** persistente und idempotente Reservierung, Commit sowie einmalige Freigabe.
- **Shop-Integration:** Happy Path, Bestands- und Zahlungsfehler, Readiness, Downstream-Ausfall, Timeouts, Event-Publikationsfehler, Commit-Retry, Kompensation, Providerpropagation und Präsentationsmodus.
- **End-to-End:** Start des realen lokalen Stacks, Checkout über die Website und Abfrage der in PostgreSQL gespeicherten Snapshot-Kette.

Die Billing-Unit-Tests verwenden kontrollierte Provider-Doubles. Die Shop-Integrationstests ersetzen gRPC-Clients und Audit-Recorder gezielt, sodass Aufrufreihenfolge, Nichtaufrufe und terminale Zustände beobachtbar sind. Technische Fehler werden nicht nur anhand des HTTP-Status bewertet: Die Tests prüfen zusätzlich Refund-, Release- und Audit-Aufrufe. Dadurch kann eine oberflächlich korrekte Fehlerantwort keinen fehlenden SAGA-Schritt verdecken.

5.2 Automatisierte Testergebnisse

Der abschließende Release-Build wurde mit .NET10-64bit erfolgreich ausgeführt.

Testprojekt	Erfolg	Fehler	Gesamt
StoreBackend.UnitTests	2	0	2
BillingService.UnitTests	10	0	10
ShopService.IntegrationTests	16	0	16
Gesamt	28	0	28

Tabelle 3: Testergebnis des Release-Builds

Nach Abschluss wurde die Produktversion für alle Komponenten auf Version 1.0 gesetzt. Ein fehlerfreier Release-Build populierte außerdem alle notwendigen Komponenten (außer RabbitMQ) im Verzeichnis `..\MASTER \VST_OnlineStore\ReleaseBuild`, ein Zip-File mit dessen Inhalt wurde unter GIT ebenfalls deployed (`VST_OnlineStore-1.0.0-win-x64.zip`).

5.3 Nachweise der Fehlerszenarien

Fehlerfall	Sichtbares Ergebnis	Nachgewiesene interne Reaktion
Zahlung abgelehnt	HTTP 422, PAYMENT_FAILED	Separate Release-Anforderung, persistente Freigabe und terminaler Snapshot; kein Commit
Unzureichender Bestand	HTTP 409, OUT_OF_STOCK	Abbruch nach Reserve-Ablehnung; Billing und Invoice nicht aufgerufen
Invoice nicht verfügbar	Bestellung bleibt COMPLETED	Versuche 1 und 2 eingeplant, Versuch 3 ausgeschöpft, Verarbeitungsfehler auditiert; kein Refund
Warehouse-Commit fehlerhaft	HTTP 502, ROLLBACK_COMPLETED	Refund über Fassade, Lagerfreigabe und je ein Snapshot für angeforderte und abgeschlossene Kompensationen

Tabelle 4: End-to-End-Beobachtung im Präsentationsmodus

Bei der Website-Prüfung des Warehouse-Fehlers wurden 22 chronologische Snapshots unter derselben Correlation-ID aus PostgreSQL gelesen. Die Kette enthielt unter anderem `STOCK_COMMIT_FAILED`, `PAYMENT_REFUND_REQUESTED`, `PAYMENT_REFUNDED`, `STOCK_RELEASE_REQUESTED`, `STOCK_RELEASED` und den terminalen Bestellstatus. Gleichzeitig waren die providerbezogene Erstattung im Billing-Log und Release-Aufrufe in Warehouse- und Backend-Logs auffindbar.

5.4 Bewertung und verbleibende Testlücken

Die Tests decken die entscheidenden Kontrollflüsse ab, erreichen aber keine Produktionsvalidierung, außerdem werden PostgreSQL und RabbitMQ in den Shop-Integrationstests, durch kontrollierte Grenzen (MOCK's) ersetzt.

Softwareprojekte dieser Größe die länger betrieben und somit auch gepflegt werden müssen, benötigen einen konsequenten Aufbau einer Testumgebung nach ISTQB[6] und V-Modell. Diese Umgebung wächst mit den Anforderungen des Projekts auf jeder Ebene. Aufgrund der verteilten Umgebung werden zusätzlich Last-, Stress- und Penetrationstests notwendig, auch *Chaos-Engineering* und Partitions/Netzwerktests können zur Robustheit beitragen.

6 Reflexion

6.1 Gute Entscheidungen

- **Zentrale Microservice-Architektur:** Alle wesentliche Zugriffe laufen über einen zentralen Shop-Service und dieser meldet nur zum StoreProxy durch. Die Architektur vermittelt eine gute Übersichtlichkeit und kann sowohl synchron als auch dynamisch asynchrone (Teil-)Prozesse durchführen und kann orchestriert werden.
- **Die Payment-Fassade stellt eine belastbare Grenze dar:** Kein Billing-Aufrufer greift direkt auf konkrete Adapter zu. Providerwahl, Timeout, Refund und Status sind einheitlich, und neue Adapter werden ohne Änderung bestehenden Fassadencodes entdeckt.
- **Fehler sind fachlich und technisch nachvollziehbar:** Correlation-ID, strukturierte Logs, OTLP-Daten und append-only Audit-Snapshots ergänzen sich. Auch Kompensationsschritte sind eigenständige Ereignisse.
- **Fault Injection:** Setzt an den tatsächlichen Servicegrenzen an. Sie simuliert keine bloßen Frontendmeldungen, sondern löst Refund, Release, Retry und DLQ-Verhalten aus.
- **Stubs für Payment-Provider, E-Mail und DB-Client sind bereits als Adapter bzw. Strategie entworfen:** Die entsprechenden Slots für diese Komponenten sind bereits gegeben, Kapselung und Erweiterbarkeit wurden berücksichtigt.

6.2 Weniger gute Entscheidungen

1. **Verwendung des *TransactionalOutboxPattern*:** Zwischen erfolgreicher Zahlung und RabbitMQ-Publikation besteht eine *Dual-Write-Grenze*. Publisher-Confirms und Refund reduzieren das Risiko, eine persistente Outbox im BillingService würde den Übergang jedoch noch robuster machen.
2. **Containerisierte Infrastruktur:** Die PowerShell-Skripte ermöglichen einen guten Windows-Demobetrieb, sind jedoch weniger portabel als Docker Compose oder eine containerbasierte Testumgebung. PostgreSQL, RabbitMQ und Collector sollten für CI reproduzierbar gekapselt werden. Dieses Feature konnte leider nicht mehr berücksichtigt werden, war aber geplant.
3. **Infrastrukturtests früher einplanen:** Viele Fehlerlücken wurden erst durch manuelles Abschalten einzelner Services sichtbar. Frühe automatisierte Ausfalltests hätten Readiness, Timeout und Kompensation schneller zusammengeführt.
4. **Retry-Richtlinie:** Drei Invoice-Versuche erfüllen zwar die Anforderung, produktionstauglicher sind jedoch Techniken wie exponentielles Backoff mit Jitter, Metriken, Alarmierung und ein kontrollierter Redrive-Prozess für die DLQ.

6.3 Eingegangene Kompromisse

1. **Hybrid aus gRPC und Events:** Zwischen erfolgreicher synchroner Konsistenz für checkoutkritische Schritte, aber „*eventual consistency*“ bei Rechnung und Auditing, nach Voraussetzung
2. **Event Sourcing Light** Sehr gute Historie leider ohne vollständige Aggregate-Rekonstruktion; operative Daten bleiben separate Wahrheit
3. **Provider-Stubs** Deterministische Tests ohne Geldbewegung; keine Aussage über Webhooks, Provider-SLAs.(War geplant)
4. **Dateiablage für Lager und Rechnung:** Einfache, sichtbare Demonstration; keine horizontale Skalierung und eingeschränkte Parallelität, dafür gute Handhabbarkeit
5. **Pickup-Outbox statt echter E-Mail** Sichere und reproduzierbare Rechnungserzeugung; Versandzustellung wird nicht validiert, dafür vorbereitet mit *Mailkit-Package*
6. **Orchestrierte SAGA:** Ablauf und Kompensation sind zentral verständlich; der ShopService trägt dafür wie geplant mehr Prozesswissen und stellt das Zentrum dar

7. **Datenbank, E-Mail** Die Komponenten funktionieren zwar nur eingeschränkt (DB-Cluster-Instanz, E-Mails werden nur in Ordner geschrieben/kein echter SMTP-Client); Aber die Komponenten sind leistungsfähig (QuestPDF/Mailkit) und der Datenbankadapter kapselt mit EF.Core

6.4 Fazit

Die Realisierung des Online-Stores mit aktueller Technologie und Systemen zeigt auf, wie stark sich grade die Webentwicklung in den letzten 20 Jahren verändert hat. Wo früher noch eine Website mit einem DOM und Model ausreichend war, sind inzwischen wesentliche Neuerungen Stand aktueller Technik, dies schlägt sich auch in der Projektgröße und Umfang nieder. Aufgrund der zahlreichen Anwendungsfälle die durch verteilte Systeme abgebildet werden können, steigt die Anzahl der Techniken, des notwendigen Konfigurationsmanagment und Modellwissens.

Wesentlich sind allerdings die Architekturmerkmale und das technische Verständnis hinsichtlich einer Microservice-Architektur. Auch wenn diese Umsetzung exemplarisch ist, war der Lerneffekt deutlich, die Realisierung eines Online-Stores „State-of-the-Art“ stellt in der Tat eine gute Übung dar, um aktuelle Technologien, Topologien und Entwurfsmuster zu verwenden bzw. zu verstehen.

Literatur und Standards

Literatur

- [1] ChatGPT-Codex Agent, zur Erstellung des Web-Frontend's <https://chatgpt.com/de-DE/codex/>
- [2] IETF: *RFC 9457 - Problem Details for HTTP APIs*, 2023. <https://www.rfc-editor.org/rfc/rfc9457>
- [3] OpenAPI Initiative: *OpenAPI Specification 3.1*. <https://spec.openapis.org/oas/v3.1.0>
- [4] E. Gamma, R. Helm, R. Johnson und J. Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [5] C. Richardson *Microservices Patterns*. Manning, 2018, Kapitel 4: Managing transactions with sagas.
- [6] International Software Testing Qualifications Board <https://istqb.org>
- [7] EntityFrameworkCore <https://learn.microsoft.com/de-de/ef/core/get-started/overview/first-app?tabs=netcore-cli>
- [8] Microsofts 'Yet Another Reverse Proxy' für .NET <https://dotnet.github.io/yarp/>
- [9] M. Fowler: *Event Sourcing*, 2005. <https://martinfowler.com/eaDev/EventSourcing.html>
- [10] RabbitMQ: *Consumer Acknowledgements and Publisher Confirms*. <https://www.rabbitmq.com/docs/confirms>
- [11] PostgreSQL Global Development Group: *PostgreSQL 18 Documentation*. <https://www.postgresql.org/docs/18/>